VLSI Implementation of Edge Detection Chip: A Prospective Design

Harsh Raj Thakur, Prabir Saha*
Dept. of ECE National Institute of Technology Meghalaya, Shillong, Meghalaya, India
kspqa126@gmail.com, sahaprabir1@gmail.com*

2024 IEEE International Symposium on Smart Electronic Systems (iSES) | 979-8-3315-3322-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISES63344.2024.00047

***Abstract*—Edge detection is a basic function of image processing and a versatile application that includes object recognition, surveillance, and medicinal imagery. A suggested circuit improves the efficiency and flexibility of the edge detection techniques by defining its behavior using Verilog, a language for describing hardware.This paper describes the design and implementation of an edge detection circuit using a 4-bit carry look ahead adder and a 2's complement circuit for handling the negative bit,using the Verilog programming languages.Improved computing speed and accuracy are achieved by the circuit by integrating a carry Look Ahead adder and a 4-bit multiply. This paper represents the circuit design step of Verilog Code, hardware synthesis, and algorithm. Performance analysis represents how well the suggested approach works for reliability and quickly identifying edges. This research presents the possibility of hardware acceleration for real-time applications and advances edge detection approaches in digital image processing. To illustrate the results, the suggested design was simulated using Verilog coding. For the purpose of estimation of VLSI parameters, it was then synthesized in Vivado (xc7vx485tffg1157-1). The perspective insights and future motivation ideas to create energy-efficient VLSI solutions for the edge detection convolution technique are provided by this implementation result.**

***Index Terms*—4-Bit Multiplier, Carry Look Ahead Adder, CNN,VLSI**

I. Introduction

In the area of digital image processing[4], the extraction of significant features from images is highly dependent on edge detection[1]. Recognizing the edges of images helps perform various tasks like object recognition, photo segmentation, mixing of pixels, and scene comprehension to provide important information about its range. Edge detection techniques[1] are very much desired in situations where quick decision making is required because of their ability to process real-time data during hardware implementation. The design and software implementation of such edge detection circuit using Verilog, a hardware description language is well known to do the perfect digital model. In this paper vertical edge detector algorithm has been focused, where the circuit have been realized through a 4-bit carry look ahead adder [2] and 2's complement circuit[3] to improve computational performance without complicating the hardware architecture. With the help of Verilog, it is possible to describe the edge detection technique[1] at a higher level of abstraction, which speeds up the process model and verification. The importance of edge detection[1] and it's techniques in the image processing[3] field is discussed in this paper, along with the sign bit multiplier with the help of 2's complement circuit[2] has been discussed in this paper.

II. Related Work

It includes edge detection circuits[1] previous research and methods with the aim of enhancing edge detection[1] systems accuracy, efficiency and reliability. adaptability. Many approaches have been explored, and the pursuit is one of the approaches made to indicate the plentiful approaches studied. as the hardware based solutions the convolutional neural networks. Among the few techniques that are worth mentioning are CNN's[5] and some of the more traditional gradient-based approaches. The Some of the observed Sobel, Prewitt[1], and Roberts operators are among the most basic operators applied to matrices in the traditional means of edge because of this, they are more preferred in edge detection[1] due to the fact that they are simple and more effective in detecting edges by concentration on intensity gradients. These techniques may, however, have poor sound reception or sensitivity to some sounds. Flexibility of the regularised singular values to different image[6] characteristics.

Such deep learning techniques, especially CNNs[5], have seen a recent surge in their use for edge detection[1] tasks owing to advances in the field. This is because CNN[5] based models are better capable of learning features from data and hence can offer better and more robust performance even when images contain noise or variability that would normally decrease the accuracy of a model. CNN's[5] may not be the best choice for real-time applications where the resources are limited, since installing CNN's[5] on hardware platforms is often associated with high computational cost. Another form is a type of edge detection that relies on specialised hardware[6] resources on nodes for efficient output. This is a general and commonly employed style favored for real-time detection of edges and other features, utilizing tailored architectures that are well-optimized for these purposes and consume minimum power. Specific catering of particular edge detection requirements and flexible and easily extensible designs through FPGA[6] and ASIC techniques.

In conclusion, the investigation of edge detection[1] circuit has revealed a number of significant improvements in relation to the working speed, power consumption and chip space. These papers described here are some of the most significant advances in these fields and serve to provide useful data and ideas and the proposal of efficient edge detection[5] circuits for various applications.

2832-3602/24/$31.00 ©2024 IEEE
DOI 10.1109/iSES63344.2024.00047

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:24 UTC from IEEE Xplore. Restrictions apply.

III. METHODOLOGY

Convolution is a mathematical approach that combines two functions to produce a third function. In the field of signal processing and image processing[4], convolution is often used to describe how two functions, basically one of them being a signal or an image and the other being a filter or a kernel[4], are merge to create a new output. In general it is a mathematical tool, which perform sliding operation at one function (the kernel or filter)[4] over the other (the input signal or image), perform multiplication[7], [10] of their values at each overlap section, and then adding up these products. This mathematical tool is used to perform an important function like filtering[4], feature extraction,Image processing[4] and signal processing[8], as it bring out necessary information or features from the input data.

.

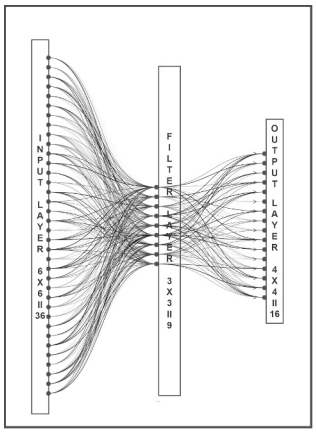


Fig. 1: Convolutional Neural Network.

In Fig 1, it demonstrate the process wherein an image with a total of 36 inputs arranged in a 6x6 grid that undergoes convolution operation with a kernel[4] matrix sized 3x3, consisting of 9 elements. Through this convolution operation, the result submit a data output of 16 elements arranged in a 4x4 grid.

A. *Algorithm Description*

This algorithm description is all about the special type of vertical edge detector filter[4] which is used to identify different types of vertical edges present in the image. This layer is needed for performing an image processing[4] task in real life because it allows the network to recognize and extract important information from edges in visual data.
The equation below represents the reduced image matrices. For example, if an image of size $n \times n$ pixels[9] is taken and the size of the filter[4] is $f \times f$, then the reduced image matrices' size will be $(n - f + 1) \times (n - f + 1)$.

$$(n \times n) * (f \times f) = (n - f + 1) \times (n - f + 1)$$

Algorithm 1: Vertical Edge Detection Algorithm

Input: P: Input image, $Filter$: Vertical edge detection filter
Output: R_{out}: Output image after applying vertical edge detection filter

Initialize R_{out} as an empty image with the reduced dimensions of P;
for *each pixel* (x, y) *in* P **do**
  Initialize sum to 0;
  for *each element* (i, j) *in the 3x3 filter* **do**
    Compute product = $P(x + i, y + j) \times \text{Filter}(i, j)$;
    Add product to sum;
  end
  Set $R_{\text{out}}(x, y)$ to the absolute value of sum;
end
Apply suitable normalization to R_{out} if required;

In the following image 6x6 data has been taken which will be convoluted with the vertical edge detector filter[4], first this filter will be mapped on the corner layer which is shown in the Fig.2, then it will slide[9] to the next layer and it will perform the operation after all this operation it will slide[9] down in the same way in each operation it will get multiplied[7], [10] with there respective cell and the final result will get summed up.

.

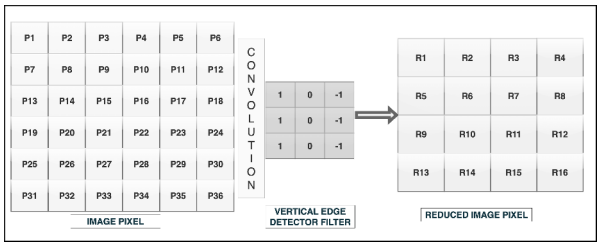


Fig. 2: Edge Detector

In Fig.3, the sliding[9] operation is appropriately demonstrated, depicting the sequential movement of the kernel matrix[4] over the input image. This sliding[9] operation executes the kernel[4] operation across the entire input image, ultimately yielding the final results of the convolution[5], [12] operation.

B. *Mathematical Calculation*

In this Part the Mathematical results for Each operation is shown in the Equation. Let $I(x, y)$ indicate the intensity of the pixel at coordinates (x, y) in the input image.

The filter i.e kernel[8] for vertical edge detection can be represented as:

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:24 UTC from IEEE Xplore. Restrictions apply.

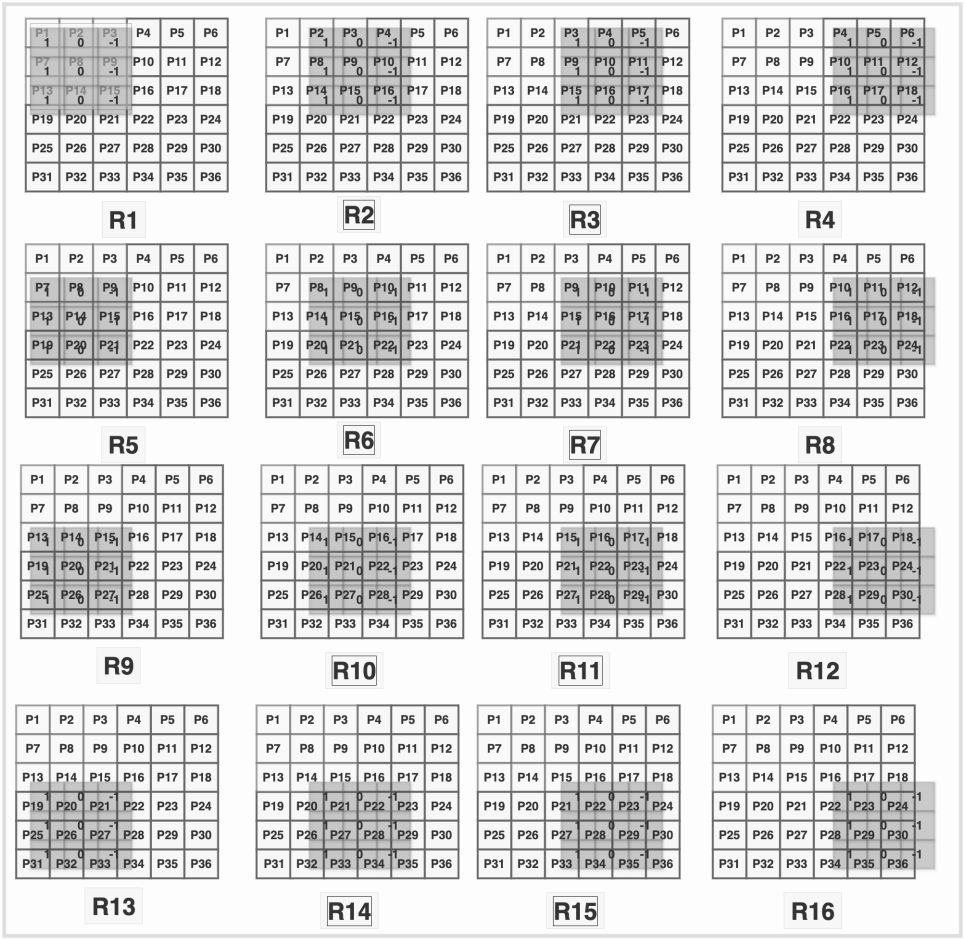


Fig. 3: Convoluted Results

$$K_v = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Then, the result of vertical edge detection[1], [6], indicated as $G_v(x, y)$, is obtained by convolving the image with this kernel:

$$G_v(x, y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} I(x + i, y + j) \times K_v(i, j)$$

where $K_v(i, j)$ shows the value of the kernel at position (i, j).

This operation computes the vertical gradient[6], [13] of the image, highlighting vertical edges[1] as positive or negative peaks depending on the direction of the edge.

The below equation is the results of Fig.3

$$\begin{aligned} R1 &= P1(1) + P2(0) + P3(-1) + P7(1) + P8(0) \\ &\quad + P9(-1) + P13(1) + P14(0) + P15(-1) \quad (1) \\ R2 &= P2(1) + P3(0) + P4(-1) + P8(1) + P9(0) \\ &\quad + P10(-1) + P14(1) + P15(0) + P16(-1) \quad (2) \\ R3 &= P3(1) + P4(0) + P5(-1) + P9(1) + P10(0) \\ &\quad + P11(-1) + P15(1) + P16(0) + P17(-1) \quad (3) \\ R4 &= P4(1) + P5(0) + P6(-1) + P10(1) + P11(0) \\ &\quad + P12(-1) + P16(1) + P17(0) + P18(-1) \quad (4) \\ R5 &= P7(1) + P8(0) + P9(-1) + P13(1) + P14(0) \\ &\quad + P15(-1) + P19(1) + P20(0) + P21(-1) \quad (5) \\ R6 &= P8(1) + P9(0) + P10(-1) + P14(1) + P15(0) \\ &\quad + P16(-1) + P20(1) + P21(0) + P22(-1) \quad (6) \\ R7 &= P9(1) + P10(0) + P11(-1) + P15(1) + P16(0) \\ &\quad + P17(-1) + P21(1) + P22(0) + P23(-1) \quad (7) \\ R8 &= P10(1) + P11(0) + P12(-1) + P16(1) + P17(0) \\ &\quad + P18(-1) + P22(1) + P23(0) + P24(-1) \quad (8) \end{aligned}$$

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:24 UTC from IEEE Xplore. Restrictions apply.

$$R9 = P13(1) + P14(0) + P15(-1) + P19(1) + P20(0) + P21(-1) + P25(1) + P26(0) + P27(-1) \quad (9)$$

$$R10 = P14(1) + P15(0) + P16(-1) + P20(1) + P21(0) + P22(-1) + P26(1) + P27(0) + P28(-1) \quad (10)$$

$$R11 = P15(1) + P16(0) + P17(-1) + P21(1) + P22(0) + P24(-1) + P27(1) + P28(0) + P29(-1) \quad (11)$$

$$R12 = P16(1) + P17(0) + P18(-1) + P22(1) + P23(0) + P24(-1) + P28(1) + P29(0) + P30(-1) \quad (12)$$

$$R13 = P19(1) + P20(0) + P21(-1) + P25(1) + P26(0) + P27(-1) + P31(1) + P32(0) + P33(-1) \quad (13)$$

$$R14 = P20(1) + P21(0) + P22(-1) + P26(1) + P27(0) + P28(-1) + P32(1) + P33(0) + P34(-1) \quad (14)$$

$$R15 = P21(1) + P22(0) + P23(-1) + P27(1) + P28(0) + P29(-1) + P33(1) + P34(0) + P35(-1) \quad (15)$$

$$R16 = P22(1) + P23(0) + P24(-1) + P28(1) + P29(0) + P30(-1) + P34(1) + P35(0) + P36(-1) \quad (16)$$

C. Edge Detection

This parts shows the one of the example how edge detection part is working in the below figure i.e Fig.4, a simple figure has been considered in which one part is showing black colour and the other part is showing white colour when it is convoluted by the edge detector filters[4] it detects the edge as it is shown in the Fig.4. .

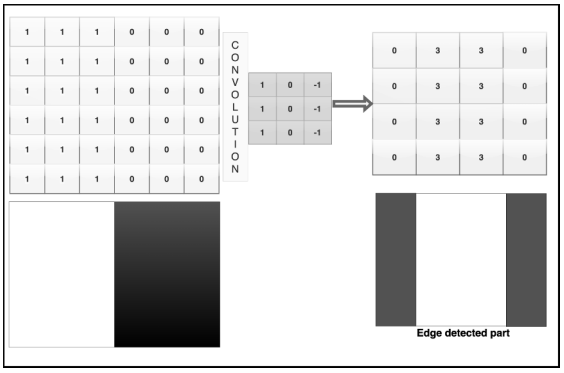


Fig. 4: Edge detected image.

white colour has been considered as a value of 1 and a black part has been considered as the value of 0.

D. Model Description

Fig.5 Represents the 2's Complement Circuit[3] which has been used in Summation Circuit with the help of XOR Gates, 4bit CLA and control signal it can be designed.

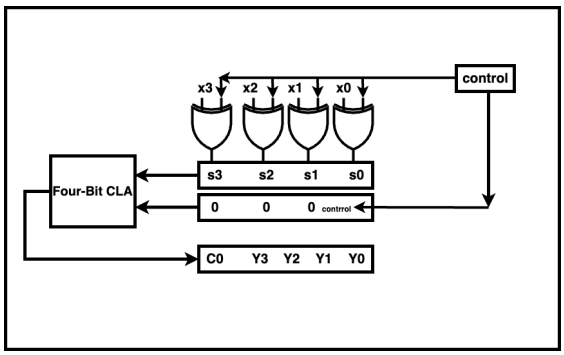


Fig. 5: 2's Complement Circuit

To implement the proposed design, summation circuits are required, which consist of a 4-bit adder, a multiplier[2], [10], and an XOR gate.

Fig.6 illustrates how the kernel[4] is multiplied by the image pixel[14]. In Fig.6, there is a 9-input sample of an image pixel[9] which will convolute with a kernel[4] of 9 input data. The matrix of the vertical edge detection kernel[4] is:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

This matrix represents the weights that the convolution process applied to nearby pixels[9]. To get the complicated result, multiply each member of this matrix by the respective element of the picture pixel[9] matrix. Then, add the results together. To address the negative[3] value in the third column of the kernel[4] input, only positive values will be considered during multiplication[2], [10].

To address the negative value in the third column of the kernel input, only positive values will be considered during multiplication[2], [10].

Modified kernel[4] input shown below in the matrix

$$\begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

After the performance of the multiplication of individual element of the kernel with the corresponding element of the image pixel[9] column-wise, the bit data will be added separately for each column. For the third column, which inferred negative values, the results will be extracted as 2's complement[3] using XOR gates and adding 0001 binary number. At last, the results of column 1, column 2, and the 2's complement[3] results for the third column will be summed up together. This approach ensures proper handling of negative values in the kernel[4] input during the convolution[5], [6] operation.

"First 4bit data" is the result of the multiplier, Least significant value since we are using 4bit multiplier so we only have to consider the LSB and same operation for 4-bit CLA the

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:24 UTC from IEEE Xplore. Restrictions apply.

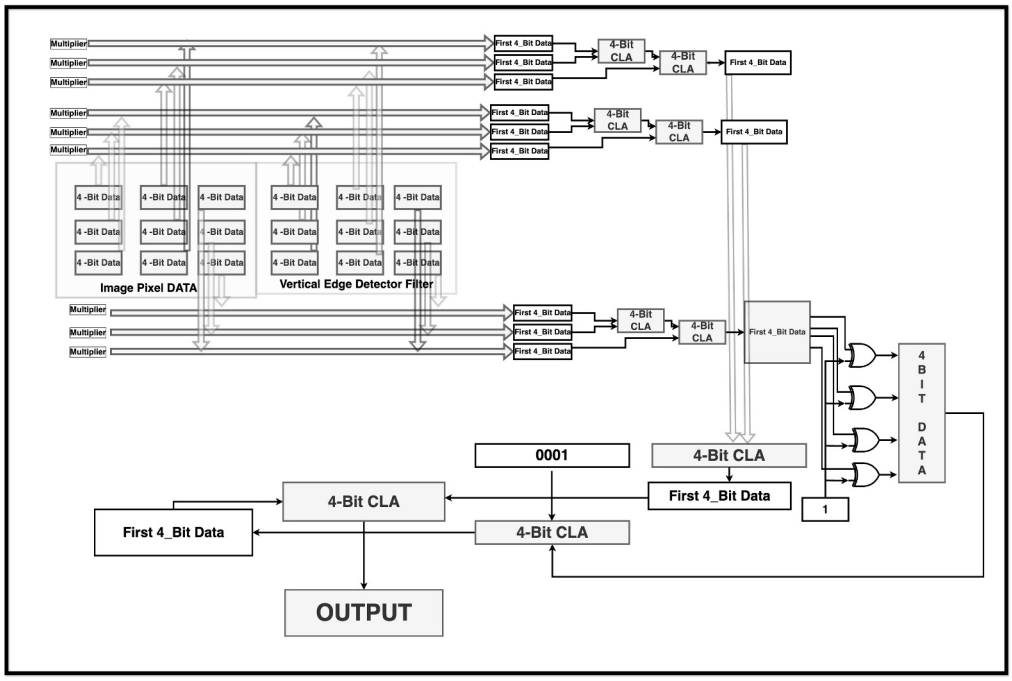


Fig. 6: Summation Circuit Architecture.

result is shown in Fig.6 by"First 4bit data".

E. Experimental Setup

In the avocation of efficient edge detection[14], a System Verilog program was detail crafted within the Xilinx environment. Employing a gradual approach, the program is structured around essential components, including a Multiplier[10] with a 4-bit adder, a 4-bit carry[2] look-ahead adder, a summation block, and finally, the essential edge detection block[14]. Each module was carefully designed to optimize performance metrics crucial to the task at hand. Through rigorous evaluation, significant parameters such as Area, Lookup Tables (LUT), and Power consumption were meticulously analyzed. This systematic approach not only ensured the effectiveness of the edge detection system[14] but also provided valuable insights into enhancing its efficiency and performance.

IV. Evaluation Metrics

In this part, a number of significant parameters are used to evaluate the performance and attributes of the proposed Vertical Edge detection block[1]. These conditions are:

1) Area calculation: The information below was learned from the results of the area calculation for the logic design of the device xc7vx485tffg1157-1.Table1. represents the utilization details for different block

   Four Bit Carry Look Ahead Adder: Utilized 4 out of 303600 available LUTs (0.0013175231% utilization), 1 out of 75900 available slices (0.001317523% utilization) and 14 out of 600 available IOBs (2.33% utilization).
   Multiplier using three 4-bit Adder: Utilized 21 out of 303600 available LUTs (0.00691% utilization), 6 out of 75900 available slices (0.00791% utilization) and 16 out of 600 available IOBs (2.67% utilization).
   Summation Block: Utilized 72 out of 303600 available LUTs (0.023% utilization), 23 out of 75900 available slices (0.03% utilization) and 76 out of 600 available IOBs (12.67% utilization).
   Edge Detection Block: It is the combination of 16 Summation Block which Utilized 1137 out of 303600 available LUTs (0.37% utilization), 336 out of 75900 available slices (0.44% utilization) and 244 out of 600 available IOBs (40.67% utilization). *xc7vx485tffg1157-1*, the following outcomes were attained:

TABLE I: Utilization Details for Different Blocks

Multiplier	Utilization		
	LUTs (%)	Slices (%)	IOBs (%)
CLA	0.00131	0.00131	2.33
Multiplier	0.00691	0.00791	2.67
Summation Block	0.023	0.03	12.67
Edge detection Block	0.37	0.44	40.67

2) On Chip Power Report: In table 2. Power details has been described for Carry Look Ahead Adder(CLA), Multiplier using 3-CLA, Summation Blocks and Edge detection Block.
3) Comparison Report: Proposed Vertical edge Detector is

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:24 UTC from IEEE Xplore. Restrictions apply.

TABLE II: On Chip Power Report

Parameter (μwatts)	Dynamic Power	Static Power	Signals Power	Logic Power	I/O Power
CLA	2.902	0.270	0.082	0.026	2.795
Multiplier (3-CLA)	3.630	0.278	0.137	0.095	3.398
Summation Blocks	6.537	0.313	1.492	0.829	4.216
Edge Detection Block	114.332	6.632	39.569	14.095	60.668

compared with Existing Model Parallel Architecture [9] in terms of hardware utilization shown in Table 3. In terms of Occupied Slices Proposed model is 6 times efficient then existing model[1] as compare to Number of LUTs Proposed model is 3.5 times efficient then existing model[1]

TABLE III: Comparison of Proposed Design and Existing Model[1] Parallel Architecture

Metric	Proposed Design	Existing Model[1]
Number of LUTs	1137	3901
Number of occupied Slices	336	1987

4) Fig 7: It's shows the implemented design on FPGA BOARD for Edge Detection Block i.e Floor Planning Layout and Input Output Layout.

.

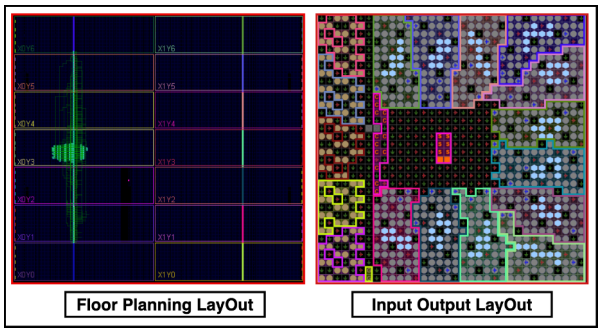


Fig. 7: Implemented Design

V. Conclusion

In conclusion, developing and testing an edge detector circuit using Verilog code has provided important new information about the circuit's performance properties. Important parameters including area, accuracy of results, and power consumption were evaluated through careful design and analysis to determine the effectiveness and efficiency of the circuit. The circuit's hierarchical layout, which included modules for the summation block, edge detection block, 4-bit carry look-ahead adder, multiplier with 4-bit adder, and summation block, allowed for a methodical optimisation process. Verilog allowed us to measure the circuit's area utilisation exactly, confirm its operation through result verification, and estimate power consumption, all of which helped us gain a complete understanding of the circuit's capabilities.

Acknowledgement

Software support to carry out the simulation of this paper has been received from the SMDP C2S programme, MeitY (grant No. EE-9/2/2021-RD-E) is gratefully acknowledged.

References

[1] V. Sanduja, Rajeev Patial, "Sobel Edge Detection Using Parallel Architecture based on FPGA", International Journal of Applied Information Systems (IJAIS), Vol. 3, No. 4, pp. 20-24, July 2012.
[2] J. Miao and S. Li, "A novel implementation of 4-bit carry look-ahead adder," 2017 International Conference on Electron Devices and Solid-State Circuits (EDSSC), Hsinchu, Taiwan, 2017, pp. 1-2, doi: 10.1109/EDSSC.2017.8126457.
[3] K. R. Fialkowski, "The 2's Complement and Other Semi-Systematic Binary Codes," in IEEE Transactions on Electronic Computers, vol. EC-15, no. 4, pp. 603-605, Aug. 1966, doi: 10.1109/PGEC.1966.264383.
[4] S. Ghosh and K. N. Chaudhury, "Kernel-Based Image Filtering: Fast Algorithms and Applications," 2019 IEEE International Conference on Image Processing (ICIP), Taipei, Taiwan, 2019, pp. 3018-3019, doi: 10.1109/ICIP.2019.8803364.
[5] R. Chauhan, K. K. Ghanshala and R. C. Joshi, "Convolutional Neural Network (CNN) for Image Detection and Recognition," 2018 First International Conference on Secure Cyber Computing and Communication (ICSCCC), Jalandhar, India, 2018, pp. 278-282, doi: 10.1109/ICSCCC.2018.8703316.
[6] M. Hailesellasie, S. R. Hasan, F. Khalid, F. A. Wad and M. Shafique, "FPGA-Based Convolutional Neural Network Architecture with Reduced Parameter Requirements," 2018 IEEE International Symposium on Circuits and Systems (ISCAS), Florence, Italy, 2018, pp. 1-5, doi: 10.1109/ISCAS.2018.8351283.
[7] N. Takagi, H. Yasuura, and S. Yajima,High-Speed VLSI Multiplication Algorithm with a Redundant Binary Addition Tree," in IEEE Transactions on Computers, vol. C-34, no. 9, pp. 789-796, Sept. 1985.
[8] W. Strauss, "Digital signal processing," in IEEE Signal Processing Magazine, vol. 17, no. 2, pp. 52-56, March 2000, doi: 10.1109/79.826412.
[9] A. Mahalanobis, R. Shilling, R. Muise, K. Hines and M. Neifeld, "Pixel resolution improvement using a sliding mask," 2016 15th Workshop on Information Optics (WIO), Barcelona, Spain, 2016, pp. 1-3, doi: 10.1109/WIO.2016.7745579.
[10] N. Divya, G. Ilakkiya, M. Dheeraj, R. Hinduja, M. G. Nathan and M. Harish, "Design and Implemetation of Combinational Logic Circuits using Open Source Tool," 2022 International Conference on Automation, Computing and Renewable Systems (ICACRS), Pudukkottai, India, 2022, pp. 82-87, doi: 10.1109/ICACRS55517.2022.10029180.
[11] S. S. Santosh, T. S. Swaroop, T. Kavya and R. Chinthala, "Complex Binary Number System-based Co-Processor Design for Signal Processing Applications," 2021 5th International Conference on Electronics, Materials Engineering & Nano-Technology (IEMENTech), Kolkata, India, 2021, pp. 1-6, doi: 10.1109/IEMENTech53263.2021.9614893.
[12] N.J.D.L. Croix, T. Ahmad and F. Han, "Enhancing Secret Data Detection Using Convolutional Neural Networks With Fuzzy Edge Detection," in IEEE Access, vol. 11, pp. 131001-131016, 2023, doi: 10.1109/ACCESS.2023.3334650.
[13] L. Hellerman, "A Measure of Computational Work," in IEEE Transactions on Computers, vol. C-21, no. 5, pp. 439-446, May 1972, doi: 10.1109/T-C.1972.223539.
[14] S. Jingcheng, W. Zhengyan and L. Zenggang, "Implementation of Sobel Edge Detection algorithm and VGA display based on FPGA," 2019 IEEE 4th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), Chengdu, China, 2019, pp. 2305-2310, doi: 10.1109/IAEAC47372.2019.8997533.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:24 UTC from IEEE Xplore. Restrictions apply.